

DEL CÓDIGO AL MUNDO REAL: MANUAL TEÓRICO

Programa de Educación en Ciencia y Tecnología
Secretaría de Extensión Universitaria
Universidad Nacional de Córdoba



Índice

Índice	2
I. Introducción a la Electrónica	3
Sistemas Eléctricos	3
Magnitudes Eléctricas	3
Aplicaciones de la Ley de Ohm:	3
Electronica Analogica	4
Ejemplos y aplicaciones	4
Electrónica Digital	4
Ejemplos y aplicaciones	5
II. Plataforma Arduino	5
Microcontroladores	5
Placa Arduino UNO	6
Sistema reactivo	7
III. Fundamentos de Programación	7
Estructura básica	7
Librerías	8
Variables	9
Definición de Pines	10
Funciones	11
Código de ejemplo (Led con delay)	11
IV. Entradas y señales	11
1. Tipos de Señales	12
2. Resistencias Pull-Up y Pull-Down	12
3. Conversor Analógico-Digital (ADC)	12
4. Función map()	12
V. Visualización con LCD	13
1. ¿Qué es LCD?	13
2. ¿Qué es I2C?	13
3. ¿Cómo la usamos?	13
VI. Sensores	15
Sensor Ultrasonico (HC-SR04)	15
Sensor DHT11(Temperatura y Humedad)	16
Fotoresistor LDR	16
Bibliografía	17
Contacto y Consultas	17

I. Introducción a la Electrónica

La electrónica es la rama de la física y la ingeniería que estudia y aplica el comportamiento y el control de los electrones en dispositivos y circuitos. Se enfoca en el diseño y uso de componentes como resistencias, transistores, diodos y capacitores, que controlan el flujo de electricidad para realizar funciones específicas, como amplificar señales, alimentar dispositivos o procesar información.

Sistemas Eléctricos

Es un conjunto de elementos interconectados que generan, transportan, distribuyen y consumen energía eléctrica. Su escala abarca desde circuitos compactos en dispositivos electrónicos hasta redes de distribución urbana.

Componentes Fundamentales:

- Fuentes de Energía: Suministran la electricidad al circuito.
- Conductores: Transportan la corriente eléctrica.
- Cargas y dispositivos de control: Consumen la energía o regulan su flujo.

Magnitudes Eléctricas

Vamos a mencionar las magnitudes eléctricas más relevantes, comparándolas con el flujo del agua:

1. **Corriente Eléctrica:** Es el flujo de electricidad que circula por un conductor, como el agua que fluye por una manguera. Se mide en amperios (A).
2. **Tensión o Voltaje:** Es la fuerza que empuja la electricidad a través de un circuito, como la presión que empuja el agua en una manguera. Se mide en volts (V).
3. **Resistencia:** Es la dificultad que encuentra la corriente para moverse por un material, como el estrechamiento de una manguera que reduce el flujo de agua. Se mide en ohmios (Ω).
4. **Potencia Eléctrica:** Es la cantidad de energía que se usa o genera en un dispositivo eléctrico, como la cantidad de agua que sale de una manguera en un tiempo determinado. Se mide en watts(W).

Ley de Ohm

La Ley de Ohm es una relación fundamental en la electrónica que establece que la corriente (I) que pasa a través de un conductor entre dos puntos es directamente proporcional a la tensión o voltaje (V) entre esos puntos e inversamente proporcional a la resistencia (R) del conductor. Matemáticamente, se expresa como:

$$I = \frac{V}{R}$$

Aplicaciones de la Ley de Ohm:

1. **Diseño de Circuitos:** Permite calcular la resistencia necesaria para un circuito dado un voltaje y corriente deseados.
2. **Diagnóstico de fallas:** Ayuda a identificar problemas en circuitos eléctricos midiendo voltaje, corriente y resistencia, y comparando los resultados con los valores esperados.

3. **Dimensionamiento de Componentes:** Es útil para seleccionar los componentes adecuados, como resistencias, en un circuito para asegurar que funcione correctamente sin sobrecargar otros elementos.

Electronica Analogica

La electrónica analógica se basa en señales continuas, lo que significa que estas pueden tomar infinitos valores dentro de un rango determinado. Esto permite representar de forma muy precisa cambios sutiles en la información.

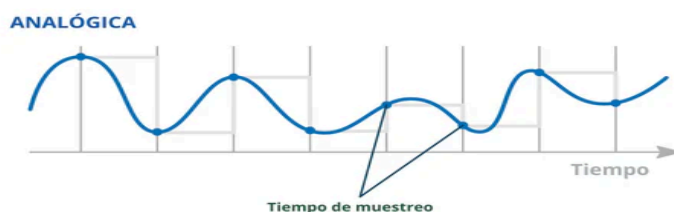


Figura 1: Representación de una señal analógica.

Imagina que tienes una perilla para ajustar el volumen de la radio. Al girarla suavemente, el sonido aumenta o disminuye de forma gradual. No se pasa de un extremo a otro de manera abrupta, sino que hay una transición suave en cada pequeño cambio. Así funciona una señal analógica: cada variación es continua y fluida, permitiendo captar matices y detalles que pueden ser muy importantes, por ejemplo, en la reproducción de sonidos o imágenes.

Ejemplos y aplicaciones

- **Control de Volumen:** Como el ejemplo de la perilla de la radio, donde el sonido sube o baja de manera progresiva.
- **Termostatos Analógicos:** Utilizan una aguja para indicar la temperatura, moviéndose de forma continua conforme cambia el ambiente.
- **Sensores de Luz o Temperatura:** Que detectan variaciones suaves en el entorno y transmiten una señal proporcional al cambio.

Electrónica Digital

La electrónica digital se fundamenta en señales discretas. Esto significa que, en lugar de tener infinitos valores, las señales digitales sólo pueden asumir ciertos valores fijos. Generalmente, estos valores se reducen a dos:

- 0 (apagado)
- 1 (encendido)

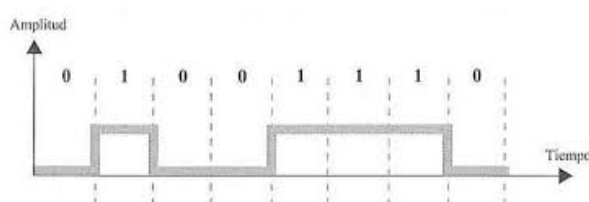


Figura 2: Representación de una señal Digital.

Para entenderlo mejor, imagina una luz que se enciende y se apaga. Cuando la luz está encendida, representa un valor (1) y cuando está apagada, representa el otro valor (0). No existe un estado intermedio en el que la luz esté "medio encendida". Esta manera de operar es la

esencia de la electrónica digital: trabajar con estados definidos que permiten un control preciso y rápido.

Ejemplos y aplicaciones

- **Luz Encendida/Apagada:** Pensa una lámpara controlada por un interruptor simple. Al presionar el interruptor, la lámpara pasa de estar apagada (0) a encendida (1) de manera inmediata, sin estados intermedios.
- **Electrónica de Computadoras y Gadgets:** Aunque los dispositivos modernos trabajan internamente con miles de millones de 0s y 1s, la idea fundamental es la misma: cada operación se traduce en estados "encendido" o "apagado". Esto permite un procesamiento rápido y fiable de la información.

II. Plataforma Arduino

Arduino es una plataforma de electrónica abierta que combina una placa de hardware programable con un entorno de desarrollo (IDE) simple, pensado para que cualquier persona pueda diseñar y construir proyectos interactivos sin necesidad de ser un experto en electrónica. En términos sencillos, una placa Arduino es una pequeña computadora especializada: recibe información del entorno a través de sensores, la procesa siguiendo el programa que nosotros escribimos y, en base a eso, controla otros dispositivos, como luces, motores o sonidos.

Su gran ventaja es que unifica en un solo ecosistema el hardware (la placa), el software (el IDE, donde se escribe el código) y una enorme comunidad que comparte proyectos, librerías y tutoriales. Esto la convirtió en la puerta de entrada más habitual al mundo de la electrónica y la programación de sistemas embebidos, tanto en la educación como en el desarrollo de prototipos.

Microcontroladores

Un microcontrolador es un sistema de cómputo completo integrado en un solo circuito integrado: incluye todo lo necesario para controlar un sistema en un dispositivo compacto. Recibe información de los sensores, la procesa y coordina una respuesta con los actuadores según lo que indique el programa. Funciona como el "Cerebro" de la máquina.

Arduino desarrolla placas de desarrollo, las cuales incorporan microcontroladores junto a otros dispositivos de soporte que articulan la utilidad del propio microcontrolador. Algunos de los tipos más conocidos de placas programables con Arduino son:

- **ATmega328P:** Se utiliza en las placas Arduino UNO y NANO. Es un microcontrolador de alto rendimiento y bajo consumo.

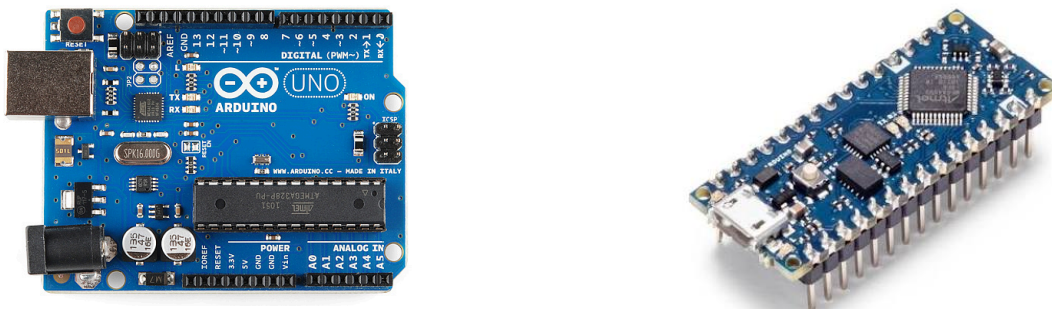


Figura 3: Arduino UNO a la izquierda, Arduino NANO a la derecha.

- **ESP32:** Es un microcontrolador de 32 bits con Wifi y Bluetooth integrados, muy usado en proyectos IoT que necesitan conectividad inalámbrica.
- **STM32:** Familia de microcontroladores de 32 bits con arquitectura ARM, de alto rendimiento y con muchos periféricos disponibles, ideal para proyectos más exigentes.
- **PIC16F887:** microcontrolador de 8 bits de la familia PIC, muy usado en la enseñanza.

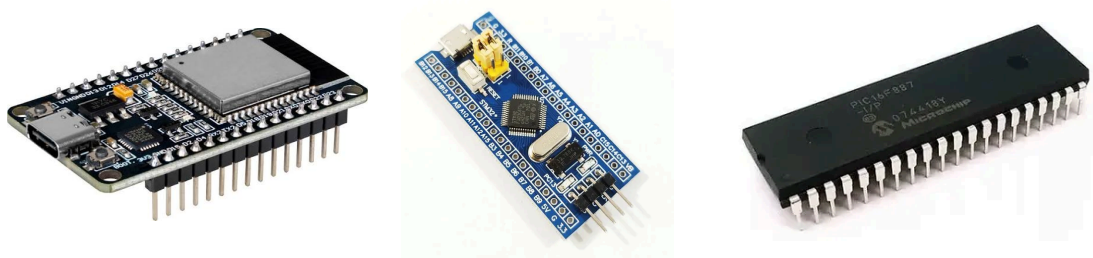


Figura 4: ESP32, STM32, PIC16F887

Placa Arduino UNO

Las placas Arduino UNO tienen 5 partes principales reconocibles:

1. **Comunicación:** A través del puerto USB se conecta la placa con la computadora. Sirve tanto para cargar el programa desde el IDE como para alimentar la placa mientras está conectada.
2. **Alimentación:** Permite alimentar la placa con una fuente externa (entre 7 y 12V), útil cuando el proyecto no está conectado a la PC por USB.
3. **Pines Digitales**(0 a 13, AREF y GND): Se usan para leer o enviar señales digitales (0 o 1). Los marcados con el símbolo ~ (3,5,6,9,10,11) también pueden generar señales PWM.
4. **Pines Analógicos**(A0 a A5): permiten leer señales analógicas provenientes de sensores (por ejemplo, un potenciómetro o un sensor de temperatura).
5. **Pines de Alimentación**(IOREF, RESET, 3.3V, 5V, GND, Vin): Proveen tensión de referencia y tierra para alimentar sensores y otros componentes externos; Vin permite alimentar la placa desde una fuente externa conectada directamente a este pin.

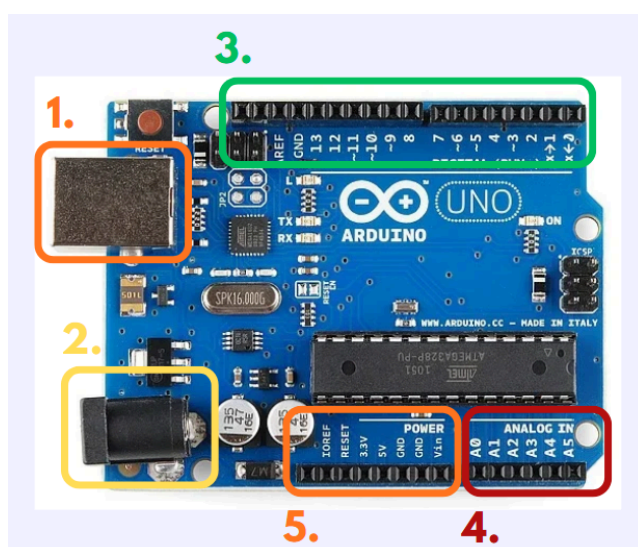


Figura 5: Partes de la placa Arduino UNO.

Sistema reactivo

Un sistema reactivo es aquel que responde de manera continua a los cambios que se producen en su entorno: Recibe información a través de sus entradas(sensores), la procesa y genera una respuesta a través de sus salidas(actuadores). Esta lógica de “leer, procesar y actuar” es exactamente la que sigue una placa Arduino durante su funcionamiento, repitiendo en ciclo mientras está encendida.

III. Fundamentos de Programación

Estructura básica

La estructura básica de todo código en arduino IDE se basa en las funciones **setup()** y **loop()**. Estas funciones tienen aplicaciones distintas pero esenciales.

El setup sirve para configurar características y establecer valores predeterminados al inicio del programa. Se utiliza entonces al principio del programa, antes de la función loop. **Se ejecuta una sola vez**, cuando comienza a correr el programa.

El loop es donde se escribe lo que deberá realizar el programa. Aquí se programan las acciones que queremos que realice nuestro sistema y cómo debe reaccionar a la información que reciba. Esta función se va a repetir siempre que el programa esté corriendo, indefinidamente hasta que se apague.

Función.	Descripción.
setup();	Se ejecuta UNA vez al iniciar el programa. Sirve para configurar pines, inicializar librerías, etc.
loop();	Se ejecuta INDEFINIDAMENTE en un ciclo o bucle. Es el “Corazón” del programa.

También tenemos el concepto de **serial**. Esto es una forma que tiene la placa de comunicarse con arduino IDE. El **monitor serial** nos permite ver los datos que está enviando la placa a la computadora, y para poder usarlo debemos configurarlo.

Por ejemplo, si tuviéramos un sensor de temperatura conectado a la placa, podríamos programarlo para que nos diga, en el monitor serial, el valor que está leyendo el sensor.

Función.	Descripción.
Serial.begin(9600);	Configura el serial para que empiece, muy importante poner el 9600 en el paréntesis pues es la velocidad a la que se comunica la placa.
Serial.println("Hola");	Imprime el mensaje , en el monitor serial, que hayamos puesto entre paréntesis, entre comillas. Las comillas indican que se trata de un texto.

Ejemplo del uso del serial: “Hola mundo”

```
void setup() {
    Serial.begin(9600);
}

void loop() {
```

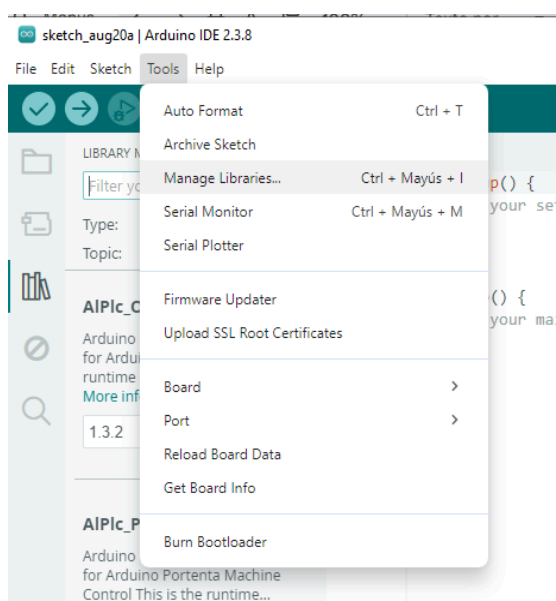
```

Serial.println("Hola mundo.");
delay(1000); //Indica que el programa espera 1000  milisegundos
antes de seguir
}
    
```

Librerías

Las librerías son paquetes de **funciones preprogramadas** que nos permiten utilizar componentes o funciones útiles para nuestro código. Para poder utilizarlas en primer lugar **debemos instalarlas** para que podamos usarlas en arduino IDE.

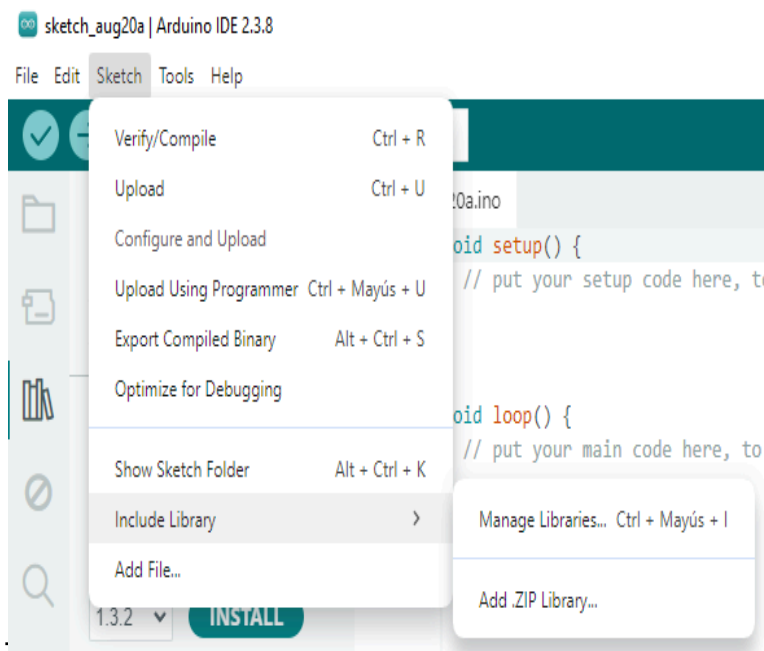
Cómo instalamos:



Vamos a **Herramientas o Tools**. De ahí a **Manage libraries o administrar librerías**.

Aquí podremos **buscar y escoger** las librerías que necesitamos para nuestros componentes o para obtener ciertas funciones convenientes.

Para poder utilizarlas en el programa ahora **debemos incluirlas** en el código.



Vamos a **sketch o programa**.

De ahí a **Include library o incluir librería**. Y ahí **elegimos** entre las librerías que tenemos instaladas.

Una vez incluida la librería en el código, debemos obtener esto por encima del setup, en las primeras líneas del código: Ejemplo con las librerías necesarias para LCD I2C.

```
#include <wire.h> //Libreria para la comunicacion I2C.
#include <LiquidCrystal_I2C.h> //Libreria para LCD I2C
```

Variables

1. **¿Qué son las variables?** Las variables son herramientas muy útiles en la programación. Nos permiten guardar valores que el programa va a usar varias veces. Las variables, como su nombre indica, pueden variar mientras el programa corre, o bien pueden ser constantes si nosotros así las definimos.
2. **¿Cómo se usan?** Para poder usar las variables primero debemos declararlas. Para ello debemos entender que hay distintos tipos de variables.

Tipo de variable.	Para qué sirve.
byte int word long	Es un tipo de variable que solo almacena números enteros, tanto positivos como negativos. Las variables byte guardan valores enteros muy pequeños. Int y word almacenan valores enteros medianos. Por otro lado, long almacena valores muy grandes enteros. Es importante saber que si intentamos asignarle un valor no entero, la variable va a almacenar solamente la parte entera del número, ignorando por completo el decimal.
float double	Pueden almacenar números con decimales. Float tiene un espacio y consumo de memoria más reducido de almacenamiento, mientras que double puede almacenar pero también consume más. Esto se ve principalmente cuando necesitamos precisión y se trabaja con números muy grandes o muy pequeños.
bool	Esta variable solo puede almacenar valores booleanos, es decir, verdadero o falso, 1 o 0. Es útil para trabajar con funciones if/else.
char	Esta variable solo puede almacenar valores de caracteres. Estos valores dependen del código ASCII, que asigna un valor numérico a cada carácter. El código ASCII se puede encontrar fácilmente buscando en internet. Es útil para trabajar con opciones a,b,c,d...
unsigned (subtipo)	Las variables unsigned no consideran el signo del valor que reciben, es decir, sólo almacenan el valor numérico absoluto y no el signo.
const (subtipo)	Las variables const no pueden cambiar su valor mientras corre el programa, son valores constantes. Mantienen el valor que les damos al principio del código.

3. ¿Cómo lo hago en el código?

Para declarar las variables en el código debemos escribir por fuera del setup y del loop de la siguiente manera: ejemplo con int:

```
unsigned int variable = 42;
```

Esa estructura de “**nombre = valor**” se puede usar por sí sola. Es decir, una vez ya declarada la variable, podemos usar “**nombre = valor2**” para asignarle un nuevo valor en alguna otra parte del código. Por ejemplo:

```
variable = 89;
```

Aclaración importante: Podemos declarar y asignar valores a variables dentro o fuera de las funciones `loop` o `setup`. Pero debemos saber que si declaramos variables, por ejemplo, dentro del `loop`, solamente se podrán usar dentro de esa función. Esto aplica para cualquier caso en que declaramos una variable dentro de una función, el uso de esa variable queda limitado dentro de la función. Es por eso que, dependiendo el caso, es recomendable declarar las variables por fuera de todas las funciones, de modo tal que funcionen como variables globales que puedan ser usadas por todo el código en todo momento.

4. Ejemplos varios:

```
// Definición de variables
float temperatura = 0;
float humedad = 0;

// 8 bits
bool estadoLED = false;
byte nivelBrillo = 128;
char letra = 'A';
unsigned char letraMayuscula = 'B';

// 16 bits
word capacidadTanque = 50000;
unsigned int valorMaximo = 1000;
int temperatura = 25;

// 32 bits
long distancia = 150000;
unsigned long tiempo = 60000;

// punto flotante
float temperaturaDecimal = 23.5;
double presionAtmosferica = 1013.25;
```

Dato interesante: Los comentarios que podemos leer clasifican las distintas variables por el uso de memoria que tienen. Las de menor espacio son las de 8 bits, las que menos datos pueden almacenar. Luego las de 16 bits se suelen usar para valores numéricos pequeños o medianos. Las de 32 bits pueden guardar datos más grandes. Finalmente las punto flotante pueden guardar datos numéricos no enteros, incluyendo los decimales.

Definición de Pines

La placa Arduino UNO posee distintos pines. Para poder utilizar los mismos, debemos aprender a definirlos en nuestro código. Esto es esencial para configurar los pines que vamos a usar.

Para definir los pines utilizaremos la creación de variables. Los pines que queramos definir pueden hacerse de dos formas, como muestra el ejemplo:

```
const int pinLed = 9;

#define pinPot A0;
```

El término **#define** no lo vimos en la parte de variables, pero funciona del mismo modo que la declaración `const int`, con la diferencia que nos permite asignarle un nombre textual a la variable en lugar de un valor numérico.

Una vez ya definidos los pines, podremos utilizar las variables que los representan. La variable `pinled` nos dice que en el pin número 9 estaremos conectando un led. Por otro lado, la variable `pinpot` nos indica que conectaremos el potenciómetro al pin A0.

Más adelante, con entradas y salidas, aprenderemos a utilizar los pines que hemos definido aquí.

Funciones

Las funciones son herramientas que nos permiten ejecutar secuencias varias veces con agilidad, sin tener que copiar toda la secuencia otra vez en el código del `loop`. Estas funciones se realizan por fuera del `loop` y del `setup`, pues siguen la misma lógica que las variables, su uso queda limitado al espacio donde se las declaró.

Para poder crear estas funciones escribiremos algo como el siguiente ejemplo:

```
//Funciones

void secuencial() {
    for (int i = 0; i < 4; i++) {
        digitalWrite(ledPins[i], HIGH); //Enciende el LED
        delay(500); //Espera 500 ms
        digitalWrite(ledPins[i], LOW); //Apaga el LED
        delay(500); //Espera 500 ms
    }
}

void secuencia2() {
    for (int i = 3; i >= 0; i--) {
        digitalWrite(ledPins[i], HIGH); //Enciende el LED
        delay(500); //Espera 500 ms
        digitalWrite(ledPins[i], LOW); //Apaga el LED
        delay(500); //Espera 500 ms
    }
}
```

Estas dos funciones de ejemplo hacen parpadear leds de maneras distintas.

Luego para poder usar estas funciones debemos “llamarlas” dentro del código. Para esto debemos poner el nombre de las funciones, por ejemplo, en el `loop`, del siguiente modo:

```
void loop() {
    secuencia2();
}
```

Con esto ya se llama a la función y comienza a ejecutarse.

Código de ejemplo (Led con delay)

Este es un ejemplo similar al que dimos antes, pero completo con el setup y loop.

```
1
2  const int ledpin = 9;
3
4  void sec1(){
5
6      digitalWrite(ledpin, HIGH); // Prenderá el led conectado al pin 9.
7      delay(1000); // Delay de 1 segundo.
8      digitalWrite(ledpin, LOW); // Apagará el led conectado al pin 9.
9      delay(1000); // otro delay.
10
11  }
12
13  void setup() {
14
15      pinMode(ledpin, OUTPUT); // configuración de salida para el pin 9.
16
17  }
18
19  void loop() {
20
21      sec1(); // llamamos la función.
22
23  }
24
```


IV. Entradas y señales

1. Tipos de Señales

En la placa Arduino UNO se distinguen tres tipos de señales:

- **Digital:** Señal discreta de dos estados: **HIGH** (1 lógico / 5 V) y **LOW** (0 lógico / 0 V). Utilizada en entradas y salidas binarias (pulsadores, relés, LEDs).
- **PWM (*Pulse Width Modulation*):** Emulación de señal analógica mediante modulación de ancho de pulso en un rango de **8 bits (0 a 255)**. Permite variar la potencia media entregada a componentes como motores o regular la intensidad lumínica.
- **Analógica:** Señal continua de tensión (0 a 5 V) que el microcontrolador digitaliza en un rango de **0 a 1023**. Se emplea para la lectura de sensores continuos y potenciómetros.

2. Resistencias Pull-Up y Pull-Down

Evitan el estado de *alta impedancia* (pin flotante) asegurando un nivel lógico definido cuando una entrada no está activada:

Configuración	Conexión en reposo	Estado en reposo (sin presionar)	Estado activo (al presionar)
Pull-Up	A 5 V (VCC)	HIGH (5 V / 1 lógico)	LOW (0 V / 0 lógico)
Pull-Down	A GND (0 V)	LOW (0 V / 0 lógico)	HIGH (5 V / 1 lógico)

Nota técnica: El microcontrolador ATmega328P de Arduino UNO incluye resistencias internas de *Pull-Up* (~20 kΩ a 50 kΩ) configurables mediante software (`pinMode(pin, INPUT_PULLUP)`).

3. Conversor Analógico-Digital (ADC)

El ADC integrado convierte los niveles de tensión analógica continuos (0 a 5 V) presentes en los pines **A0** a **A5** en valores numéricos discretos de **10 bits (0 a 1023)** para su procesamiento en el código.

4. Función map ()

Permite realizar una interpolación lineal para reescalar un valor desde un rango de origen hacia un rango de destino.

Sintaxis:

```
map(valor, min_origen, max_origen,min_destino, max_destino);
```

Ejemplo:

```
map(humedad, 0, 1023, 0, 255);
```

- **humedad (Valor):** Variable de entrada que contiene el dato analógico a transformar.
- **0, 1023 (Rango de origen):** Límites mínimo y máximo de la escala analógica actual (ADC de 10 bits).
- **0, 255 (Rango de destino):** Límites mínimo y máximo de la escala de salida deseada (PWM de 8 bits).

V. Visualización con LCD

1. ¿Qué es LCD?

LCD significa Liquid Crystal Display. Es una pantalla que nos permite visualizar datos o mensajes según el programa que hayamos creado. Funciona con una luz que se encuentra detrás de un líquido, el cual cambia sus propiedades con la electricidad. Estos cambios son controlados por el programa para producir formas de caracteres, los cuales bloquean la luz que viene de atrás para formar estas figuras visibles.

El modelo que utilizamos nosotros posee 2 filas y 16 columnas. Es decir, dos renglones con 16 letras de espacio. Si queremos escribir mensajes más largos que 16 letras por cada renglón, tendremos que recurrir a otras funciones que ya veremos. Muy importante recalcar que el sistema lee las filas desde 0 a 1 y las columnas desde 0 a 15.

2. ¿Qué es I2C?

I2C (o I cuadrado C) es un protocolo de comunicación que simplifica enormemente el uso de la pantalla LCD. Nos permite reducir drásticamente la cantidad de pines necesarios para conectar la LCD a la placa arduino, resultando nada más en 2 cables de alimentación (GND y VCC) y los cables de comunicación SDA y SCL. SDA es el cable que envía y recibe datos, mientras que SCL es el clock o reloj que marca el paso al cual la pantalla va actualizando mientras corre el programa.



3. ¿Cómo la usamos?

Primero vamos a necesitar la librería del LCD e I2C. Para ello iremos a herramientas, luego incluir librerías, y buscaremos "LiquidCrystal_I2C". instalaremos las versiones de Frank o Brabander, o alguna otra compatible.

La librería incluye varias funciones, aquí varias de ellas muy útiles:

Función.	Descripción.
<code>lcd.init();</code>	Inicializa la pantalla.
<code>lcd.backlight();</code>	Enciende la retroiluminación (Luz trasera para

	mejor contraste).
lcd.noBacklight();	Apaga la retroiluminación.
lcd.setCursor(columna, fila);	Dicta desde dónde se empieza a escribir el mensaje. (columnas 0-15 y filas 0-1).
lcd.print(valor);	Imprime el texto o valor numérico que le hayamos dado, desde donde hayamos puesto el cursor.
lcd.clear();	Borra todo lo que haya escrito en la pantalla hasta ese momento.
lcd.blink();	Hace que el cursor empiece a parpadear.
lcd.noBlink();	Detiene el parpadeo del cursor.
lcd.scrollDisplayLeft(); lcd.scrollDisplayRight();	Hace que el texto que hay puesto se vaya moviendo hacia la derecha (right) o izquierda (left).

4. Código de ejemplo: Cronómetro en LCD.

```

LiquidCrystal_lcd(0x27,16,2); //Configura LCD con I2C (0x27) y
// el tamaño 16 Col 2 Fil.

unsigned long startTime; // Variable para el tiempo de inicio.
unsigned long elapsedTime; // Variable para el tiempo que pasó.

void setup() {

  lcd.init();
  lcd.backlight();
  startTime = millis() // Esto almacena el tiempo de inicio.
  wire.begin() // Inicia la comunicación del I2C.
}

void loop() {

  elapsedTime = millis() - startTime; // Calcula el tiempo que pasó.

  int minutes = elapsedTime/60000 // Calcula los minutos que pasaron.
  int seconds = (elapsedTime % 60000)/1000; // Calcula segundos.

  lcd.setCursor(0,0); // Columna 0, Fila 0.
  lcd.print("Cronómetro");
  lcd.setCursor(0,1); // Columna 0, Fila 1.
  lcd.print(minutes);
  if(seconds<10){
    lcd.print(0);
  };
  lcd.print(seconds);
}

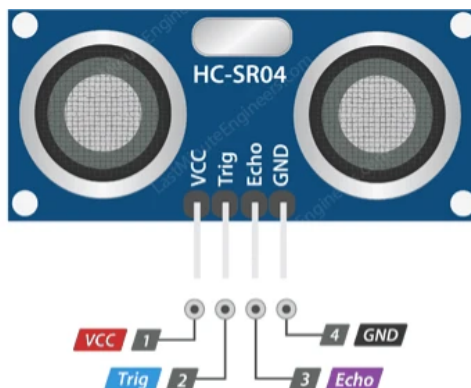
```

Este ejemplo es un cronómetro, mostrando en el display la cuenta de segundos y minutos que pasaron desde que se inició el programa.

VI. Sensores

Sensor Ultrasonico (HC-SR04)

El HC-SR04 mide distancias sin contacto físico emitiendo pulsos de ultrasonido (40 kHz) y midiendo el tiempo transcurrido hasta recibir el eco reflejado por un obstáculo.



Pin	Funcion
VCC	Alimentación Continua de 5V.
Trig	Disparo: Recibe un pulso en HIGH de 10us para iniciar la rafaga.
Echo	Eco: Permanece en HIGH un tiempo proporcional a la distancia de ida y vuelta.
GND	Masa Comun.

Fórmula de cálculo: Considerando la velocidad del sonido en el aire como 0.034 cm/ μ s y dividiendo entre 2 por el trayecto de ida y vuelta:

```
distancia_cm = (tiempo_microsegundos * 0.034) / 2;
```

Código de implementación HC-SR04:

```
const int trigPin = 9;
const int echoPin = 10;

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

void loop() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duracion = pulseIn(echoPin, HIGH);
  float distancia = (duracion * 0.034) / 2;
```

```

Serial.print("Distancia: ");
Serial.print(distancia);
Serial.println(" cm");

delay(500);
}
    
```

Sensor DHT11(Temperatura y Humedad)

Sensor digital compuesto por un termistor NTC y un sensor capacitivo para humedad con circuito de calibración interno digitalizado.

1. **Rango de humedad:** 20% a 90% RH (precisión $\pm 5\%$).
2. **Rango de temperatura:** 0°C a 50°C (precisión $\pm 2^\circ\text{C}$).
3. **Frecuencia de muestreo:** 1 Hz (una lectura por segundo).

Código de implementación DHT11:

```

#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

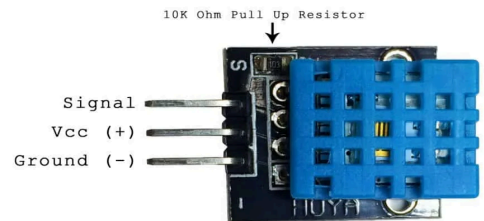
void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float humedad = dht.readHumidity();
  float temperatura = dht.readTemperature();

  if (isnan(humedad) || isnan(temperatura)) {
    Serial.println("Error al leer el sensor DHT11");
    return;
  }

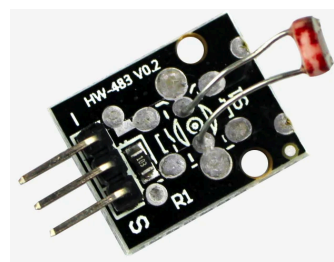
  Serial.print("Humedad: ");
  Serial.print(humedad);
  Serial.print(" % | Temperatura: ");
  Serial.print(temperatura);
  Serial.println(" *C");

  delay(2000);
}
    
```



Fotoresistor LDR

Una fotorresistencia varía su valor óhmico en función de la radiación luminosa incidente: a mayor luminosidad menor resistencia. Se integra en un circuito divisor de tensión con una resistencia fija de 10 kΩ para traducir la variación en un nivel de tensión leíble por un pin analógico.




```
const int pinLDR = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int valorLuz = analogRead(pinLDR);
  Serial.print("Nivel de luz (0-1023): ");
  Serial.println(valorLuz);
  delay(500);
}
```

Bibliografía

- **Boylestad, R. L., & Nashelsky, L.** (2018). *Electrónica: Teoría de circuitos y dispositivos electrónicos* (11.ª ed.). Pearson Educación.
- **Floyd, T. L.** (2016). *Fundamentos de sistemas digitales* (11.ª ed.). Pearson Educación.
- **Arduino Project.** (s.f.). *Arduino Reference: Language, Functions and Standard Libraries*. Disponible en: <https://www.arduino.cc/reference/en/>
- **Adafruit Industries.** (s.f.). *Adafruit DHT Sensor Library Documentation*.

Contacto y Consultas

Para resolver dudas sobre los contenidos de este manual o solicitar información acerca de futuras actividades y capacitaciones, puede comunicarse a través del siguiente canal oficial:

- **Correo electrónico:** educacionencyt@extension.unc.edu.ar
- **Área:** Programa de Educación en Ciencia y Tecnología — Secretaría de Extensión (UNC)